

Conversational Channels Cheat Sheet

Irurueta Docs · Guides & References · AI · Conversational Channels · Cheat Sheet

Baseline written 2026-10-01: LangChain 1.4.3 (langchain-core 1.6.6, langgraph 1.2.12), FastAPI 0.142.2 (Python 3.12), Spring AI 2.0.1 on Spring Boot 4.1.1 (Java 21), Express 5.2.1, AI SDK ai 7.x / @ai-sdk/react 4.x, AG-UI 1.0, aiortc 1.15.0, slxmpp 1.17.0, Smack 4.5.0, Teams SDK 2.1.x, Microsoft 365 Agents SDK 1.x, slack-bolt 1.30.0, python-telegram-bot 22.8 (Bot API 10.3), twilio 9.11.2. Policy statements dated when read; re-check before launch. Snippets are illustrative, not compiled.

CORE + ADAPTERS

```
ask(conversationId, userId, text)
-> stream of {token | citation | done}
```

One channel-agnostic core (LangChain create_agent + retriever + Postgres checkpointor, or Spring AI ChatClient + RAG advisor + JDBC memory). Each channel is a thin adapter. **Adapter duties**

- Map channel user/thread ids to userId / conversationId (drives memory and ACL filter); memory key userId:conversationId
- Map token stream to what the channel can do: native stream, edit-in-place, typing indicator, or whole message
- Render citations natively: links, cards, buttons
- Verify signature first, ack fast, dedupe on event id, then call the core
- Hand off to a human; per-tenant rate limits; async job for slow answers

CHANNEL MATRIX

Channel	Streaming	Gate
SSE / HTTP	native tokens	own auth
WebSocket	duplex	own auth
Widget	SSE	CORS, CSP
WebRTC DC	messages	TURN
XMPP	XEP-0308 edits	server
Teams	1:1 chats only	Entra, admin consent
WhatsApp	whole messages	24 h window, templates, AI policy
Apple MfB	whole messages	MSP + Apple approval
Slack	native stream	app review (public)
Telegram	draft preview	bot token
Google Chat	one reply / edits	Workspace admin; 30 s
RCS	whole messages	partner + carrier launch

SSE VS WEBSOCKET VS NDJSON

- SSE** text/event-stream: server->client, auto-reconnect, Last-Event-ID; EventSource is GET-only, no headers
- WebSocket**: duplex, you reconnect/resume; use for cancel, typing
- NDJSON**: one JSON per line via fetch reader, any method/headers

Always: keep-alives, buffering off (X-Accel-Buffering: no, Cache-Control: no-cache), cancel upstream on disconnect, identity from verified credential.

STREAMING SNIPPETS

FastAPI (fastapi.sse, 15 s keep-alive built in)

```
@app.post("/v1/chat", response_class=EventSourceResponse)
async def chat(req):
    async for ev in ask(agent, req.cid, uid, req.text):
        yield ServerSentEvent(data=asdict(ev), event=kind(ev))
```

Spring WebFlux

```
@PostMapping(produces=TEXT_EVENT_STREAM_VALUE)
Flux<ServerSentEvent<CoreEvent>> chat(...) {
    return core.ask(cid, uid, text)
        .map(e -> ServerSentEvent.builder(e).build()); }
```

Node / Express

```
res.on("close", () => abort.abort());
Readable.fromWeb(upstream.body).pipe(res);
```

Resume: send id:, honour Last-Event-ID. Spring: STOMP if a broker exists.

AI SDK UI MESSAGE STREAM

SSE of JSON parts, ends data: [DONE].

```
start{messageId} start=step
text-start{id} text-delta{id,delta}
text-end{id}
source-url{sourceId,url,title}
source-document{sourceId,mediaType,title}
data={data} tool-input=* tool-output-available
finish=step finish | abort{reason} | error{errorText}
```

Client: useChat from @ai-sdk/react. Server emits parts from FastAPI or Spring. Citations map to source-url.

AG-UI 1.0 EVENTS

Group	Events
Lifecycle	RUN_STARTED(threadId,runId), RUN_FINISHED, RUN_ERROR, STEP_STARTED/FINISHED
Text	TEXT_MESSAGE_START/CONTENT(delta)/END/CHUNK
Tools	TOOL_CALL_START/ARGS/END/RESULT/CHUNK
State	STATE_SNAPSHOT, STATE_DELTA(RFC 6902 patch), MESSAGES_SNAPSHOT
Other	ACTIVITY_*, REASONING_*, RAW, CUSTOM

Choose AI SDK for chat UIs; AG-UI for agents with shared state, tool UIs, interrupts.

READY-MADE UIs AND WIDGET

- assistant-ui** (React) on AI SDK / AG-UI streams
- Chainlit** (Python app UI), **Open WebUI**, **LibreChat** (full products, OpenAI-style API in front of the core)
- Widget**: custom element + Shadow DOM, fetch SSE reader, textContent not innerHTML, sanitise markdown, strict CSP, CORS allow-list, short-lived token, ARIA live region, keyboard + focus

XMPP BOT

XEP	Use
0030	Service discovery
0045	MUC rooms, answer on mention
0085	composing while generating
0114	Component protocol
0184	Delivery receipts
0198	Stream management (resume)
0199	Ping keep-alive
0280	Message carbons
0308	Last message correction = pseudo-streaming (bounded rate)
0313	MAM history

slxmpp (Python), Smack (Java), xmpp.js; servers ejabberd, Openfire. TLS required, JID = identity.

TEAMS: WHICH ROUTE

Route	You host	Pick when
Teams SDK	bot + model call	Teams-native features, groups, own RAG; streams in 1:1
M365 Agents SDK	agent + model call	one agent, several Microsoft channels
Copilot Studio	MCP server only	fast, low code, governance
Declarative agent	MCP server / API	users live in M365 Copilot (licence)

Bot Framework SDKs retired. Needs Azure Bot registration, Entra app, manifest, SSO; citations as Adaptive Cards; proactive messages need a stored conversation reference.

WHATSAPP BUSINESS

- Cloud API: POST /{phone-number-id}/messages, messaging_product: whatsapp; no streaming, send whole message or chunks
- Webhook: GET verify-token handshake; every POST verify HMAC-SHA256 on raw body; 200 fast; dedupe message id
- 24 h window**: free text only within 24 h of the user's last message; outside it only an approved **template** (utility / authentication / marketing)
- Interactive: up to 3 reply buttons or a list; citations as text lines
- Opt-in required; per-message pricing (official page, never hard-code)
- AI policy** (terms effective 2026-01-15, text not re-read): business-scoped RAG over own content is the case reported as allowed; general-purpose assistants reported as restricted. Read current terms
- Alternative: Twilio WhatsApp (sandbox, other signature scheme)

APPLE MESSAGES FOR BUSINESS

Business-initiated, hosted via a **Messaging Service Provider**; no direct webhook from Apple.

- Apple Business Register: organisation, verify
- Messages for Business account: name, logo, hours; Apple review (days)
- Connect to MSP console; register your webhook there
- Test with MSP tools, submit for Apple review, launch

Entry points: Messages button, Message Suggest. Features: quick replies, list picker (returns an id), Apple Pay, authentication (verified userId). Options: existing MSP, CCaaS that is an MSP, never become an MSP.

SLACK, TELEGRAM, GOOGLE CHAT

	Stream	Budget
Slack	chat_stream() start/append/stop; status "is thinking"; Events API or Socket Mode	ack ~3 s
Telegram	sendMessageDraft (private chats), throttle ~0.5 s, text[:4096], then sendMessage; long polling or webhook	prompt
Google Chat	no token stream: reply once or update message via Chat API; verify JWT	30 s

Ids: Slack team_id+channel+thread_ts; Telegram chat.id, from.id; Chat space.name, thread.name. Tokens from env/vault.

RCS BUSINESS MESSAGING

Needs a partner/aggregator and carrier launch approval; agent verification and brand review. Rich cards, suggested replies, whole messages (no token streaming). Plan lead time; fall back to SMS handled by the partner. Pricing set by partner and carriers.

OPERATIONS CHECKLIST

- Verify signature/token/JWT on raw body, constant time, before work
- Secrets in env/vault per channel, rotatable, never logged
- Ack before model call; dedupe on platform id; idempotent replies
- Serialise per conversation; queue survives restart
- One trace per turn: channel, pseudonymous user, event id, tokens
- Rate limits per identity/tenant, per-turn caps, daily spend cap, circuit breaker
- Pseudonymise ids; retention and deletion per data class
- Moderate input and output; refusal text; human hand-off
- Unit, replay, fault-injection tests; sandbox canary per channel
- Meter cost per channel; alert before budget ends

WEBRTC DATA CHANNEL

- Browser opens signalling WebSocket, sends bearer token; server derives userId
- Browser STUN lookup, createDataChannel("chat"), createOffer, setLocalDescription
- Offer (SDP) to server; aiortc setRemoteDescription, createAnswer
- Answer back; trickle ICE candidates both ways
- ICE checks, DTLS, SCTP; channel opens; close signalling unless renegotiating
- JSON ask / token / citation / done messages; chunk below the message size limit

TURN (coturn) needed behind symmetric NAT. Prefer WebSocket unless a media connection already exists.